STM32基础 - 中断

课程目标

1. 熟悉中断基础知识，熟悉STM32的中断以及NVIC中断管理
2. 掌握EXTI外部中断原理

课程试验

利用STM32完成基本的对射式传感器计次试验

课堂引入

现在的上班族每天都会奔波于家庭与办公室之间，生活节奏很快。早八点往往是各大写字楼电梯最为繁忙的时候，忙碌的上班族为了赶上上班打卡的最后时间匆匆挤入早已拥挤不堪的电梯中，2层、3层、4层……突然大家发现电梯不动了，卡在了2个楼层的中间。所有的人都变得紧张起来，终于有人提醒赶紧拨打电梯紧急求救电话。电话响了，大楼的物业人员得到了消息，开始通知电梯维护修理人员赶赴现场，“营救”卡在楼层之间惊魂未定的上班族们。

这是一个你我生活中最为常见的景象。对于电梯维护修理和物业人员来说，这个紧急电话就是一个突发的中断事件。收到这个警报，他们就必须立刻停下自己当前手头的工作，而是赶紧去处理营救被困在电梯中的上班族。在STM32单片机的世界中也有很多类似情况，CPU也常常会扮演电梯维护修理人员的角色。

授课进程

一、中断系统

1、什么是中断?

中断的定义就是在主程序运行过程中，出现了特定的中断触发条件（中断源），使得CPU暂停当前正在运行的程序，转而去处理中断程序，处理完成后又返回原来被暂停的位置继续运行。

中断就是在正常主程序执行的某个时刻发生了中断触发条件。比如，对于外部中断来说，可以是引脚发生了电平跳变；对于定时器来说，可以是定时时间到了；对于串口通信来说，可以是接收到了数据。当这些事件发生时，情况会比较紧急。比如，外部中断来了，你如果不来处理，下一个跳变信号就跟着过来了。比如串口接收中断来了，如果你不来读取接收到的数据，那下一个数据再过来就会把原来的数据覆盖掉。所以我们希望当中断条件满足时，CPU能够立即停下当前执行的程序，转而去处理这些中断事件的程序。当这些紧急事件处理完成后，CPU还能回到原来停下的地方继续运行。这就是中断的处理流程和用途。

使用中断系统能够极大的提高程序的效率。如果没有中断系统，为了防止外部中断被忽略或者串口数据被覆盖，那主程序就只能不断的查询是否有这些事件发生，不能再干其他事情了。但有了中断系统之后，主程序就可以放心执行其他事情，有中断的时候再去处理，这样效率就会大大提升。这就好像你早上定了一个闹钟，定好之后就可以放心睡觉了，时间到了闹钟会提醒你，就相当于产生了一个中断信号。

2、中断优先级

中断优先级就是当有多个中断源同时申请中断时，CPU会根据中断源的轻重缓急进行裁决，优先响应更加紧急的中断源。

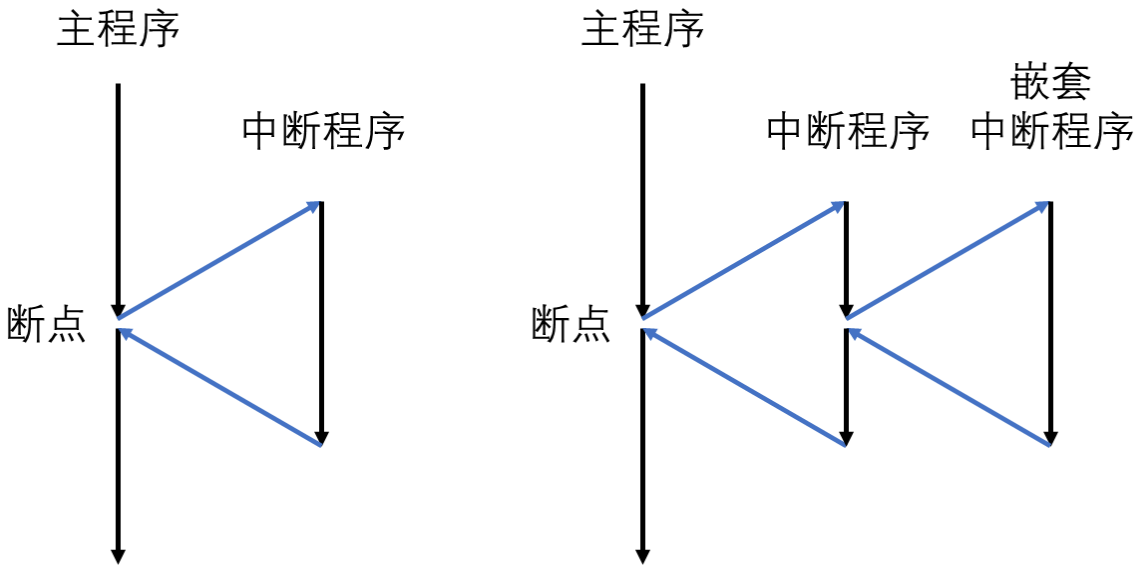
中断优先级也就是中断的紧急程度。我们可以根据程序设计的需求自己来设置，中断优先级是为了在多个中断同时申请时，判断一下应该先处理哪个。如果事件非常紧急，那就需要把优先级设置高一些，如果不是那么紧急，那就可以把优先级设置低一些，这样可以更好的安排这些中断事件，防止紧急的事件被别的中断耽误。在STM32中中断优先级还有多种分类，这个我们后续再详细来看。

3、中断嵌套

当一个中断程序正在运行时，又有新的更高优先级的中断源申请中断，CPU再次暂停当前中断程序，转而去处理新的中断程序，处理完成后依次进行返回。这种把中断程序再次中断的现象，就叫做中断嵌套。

中断嵌套也是为了照顾非常紧急的中断的，如果现在CPU已经在执行某个中断程序了。这时又发现了一个非常紧急的中断，那这个中断
就可以把当前的中断程序进行二次中断，这样新的紧急中断就可以立即被执行了。至于中断能否进行中断嵌套这个也是由中断优先级来决定的。

4、中断执行流程



左图表示中断的过程，右图表示中断嵌套的过程。

5、STM32中断

STM32有 `68` 个可屏蔽中断通道（这个是F1系列最多的中断数量，对于具体型号还是以对应手册为准，`STM32F103C8有60个`），包含EXTI、TIM、ADC、USART、SPI、I2C、RTC等多个外设。使用NVIC统一管理中断，每个中断通道都拥有 `16` 个可编程的优先等级，可对优先级进行分组，进一步设置抢占优先级和响应优先级。

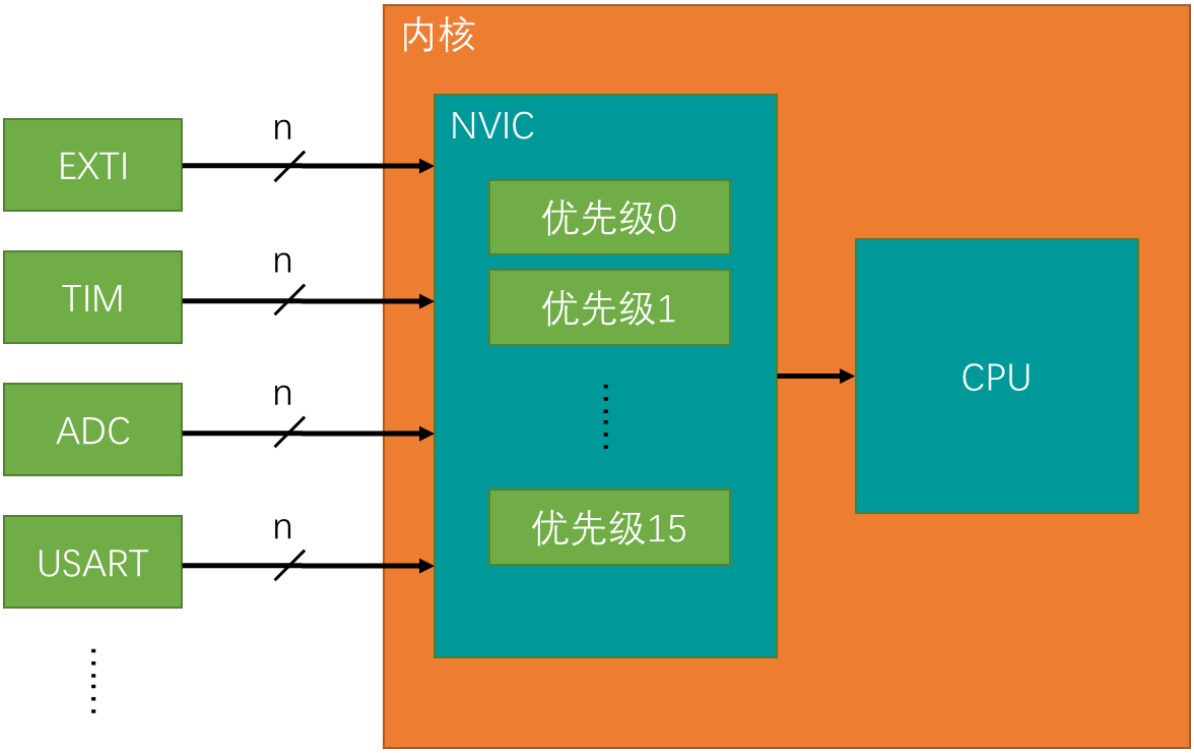
表55　其它STM32F10xxx产品(小容量、中容量和大容量)的向量表

位置	优先级	优先级类型	名称	说明	地址
	-	-	-	保留	0x0000_0000
	-3	固定	Reset	复位	0x0000_0004
	-2	固定	NMI	不可屏蔽中断 RCC时钟安全系统(CSS)联接到NMI向量	0x0000_0008
	-1	固定	硬件失效(HardFault)	所有类型的失效	0x0000_000C
	0	可设置	存储管理(MemManage)	存储器管理	0x0000_0010
	1	可设置	总线错误(BusFault)	预取指失败，存储器访问失败	0x0000_0014
	2	可设置	错误应用(UsageFault)	未定义的指令或非法状态	0x0000_0018
	-	-	-	保留	0x0000_001C ~0x0000_002B
	3	可设置	SVCall	通过SWI指令的系统服务调用	0x0000_002C
	4	可设置	调试监控(DebugMonitor)	调试监控器	0x0000_0030
	-	-	-	保留	0x0000_0034
	5	可设置	PendSV	可挂起的系统服务	0x0000_0038
	6	可设置	SysTick	系统嘀嗒定时器	0x0000_003C
0	7	可设置	WWDG	窗口定时器中断	0x0000_0040
1	8	可设置	PVD	连到EXTI的电源电压检测(PVD)中断	0x0000_0044
2	9	可设置	TAMPER	侵入检测中断	0x0000_0048
3	10	可设置	RTC	实时时钟(RTC)全局中断	0x0000_004C
4	11	可设置	FLASH	闪存全局中断	0x0000_0050

图中灰色的是内核中断，一般这些中断都比较高层，我们一般用不到。其他白色部分就是STM32外设的中断，这些部分就是我们需要关注，可能会用到的。

6、NVIC基本结构

NVIC（Nest Vector Interrupt Controller）称为嵌套中断向量控制器。在STM32中它是用来统一分配中断优先级和管理中断的。NVIC是一个内核外设，**它是CPU的小助手**。对照上面的引脚定义图可以看到在STM32的中断是非常多的，如果把这些中断全都接到CPU上，那CPU还得引出很多线进行适配，这样设计上就很不方便。此外如果很多中断同时申请，这么多的中断就会产生拥堵，导致CPU很难处理。所以中断分配的任务最好还是有其他人来处理比较好。NVIC就是这样一个设计来负责管理中断的，它有很多输入口。再多的中断线路都可以接进来。比如下图接入了EXTI、TIM、ADC以及USART等。



图中线上画了个斜杠，上面写个n。它的意思是一个外设可能会同时占用多个中转通道，所以这里有n条线。NVIC只有一个输出口，NVIC根据每个中断的优先级分配中断的先后顺序，之后通过右边这个输出口告诉CPU该处理哪个中断。至于中断先后顺序分配的任务，CPU就不需要管了。

7、NVIC中断优先级分组

NVIC的中断优先级由优先级寄存器的4位（0~15）决定，这4位可以进行切分，分为高n位的抢占优先级和低4-n位的响应优先级。这个优先级的数是 值越小，优先级越高 。抢占优先级高的可以**中断嵌套**，响应优先级高的可以**优先排队**，抢占优先级和响应优先级均相同的按**中断号**排队。

分组方式	抢占优先级	响应优先级
分组0	0位，取值为0	4位，取值为0~15
分组1	1位，取值为0~1	3位，取值为0~7
分组2	2位，取值为0~3	2位，取值为0~3
分组3	3位，取值为0~7	1位，取值为0~1
分组4	4位，取值为0~15	0位，取值为0

优先级规则

1. 优先级值越小，优先级越高。
2. 如果不设置优先级，则默认优先级为0。
3. 先比较抢占优先级。抢占优先级高的可以打断抢占优先级低的。
4. 若抢占优先级一样，再比较响应优先级。但是响应优先级不会导致中断嵌套。
5. 若抢占优先级一样的同时挂起，则优先处理 响应抢占优先级高 的。
6. 若挂起的优先级（抢占和响应）都一样，则查找中断向量表， 值小的先响应 。

二、EXTI外部中断

1、基本介绍

EXTI（Extern Interrupt）外部中断。EXTI可以监测指定GPIO口的电平信号，当其指定的GPIO口产生电平变化时，EXTI将立即向NVIC发出中断申请，经过NVIC裁决后即可中断CPU主程序，使CPU执行EXTI对应的中断程序。这个就是EXTI的基本功能，简单来说就是引脚电平的变化会申请中断。

EXTI支持的触发方式包含**上升沿**、**下降沿**、**双边沿**、**软件触发**四种。

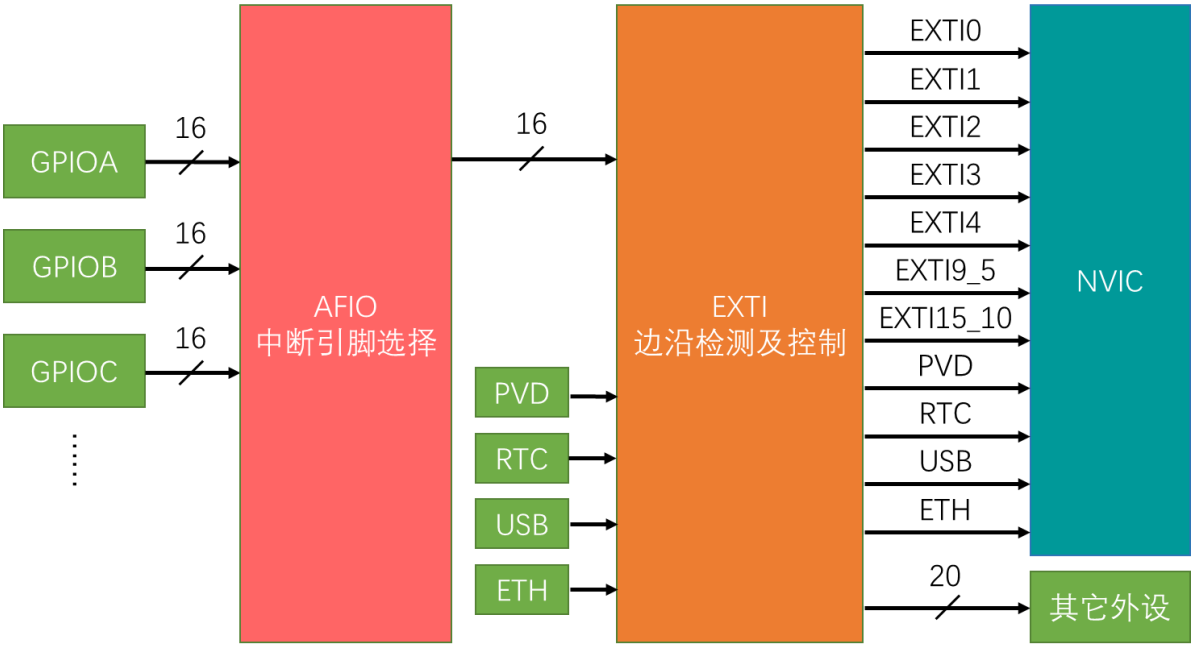
EXTI支持的GPIO口：包含所有GPIO口，但**相同的Pin不能同时触发中断。**意思就是比如PA0和PB0不能同时用，或者PA1、PB1、PC1这样的端口，只要GPIO_Pin是一样的，就只能选一个作为中断引脚。如果要使用多个中断引脚，要选择不同PIN的引脚。比如PA6和PA7、PA9和PB15、PB0和PB1这样的都可以。

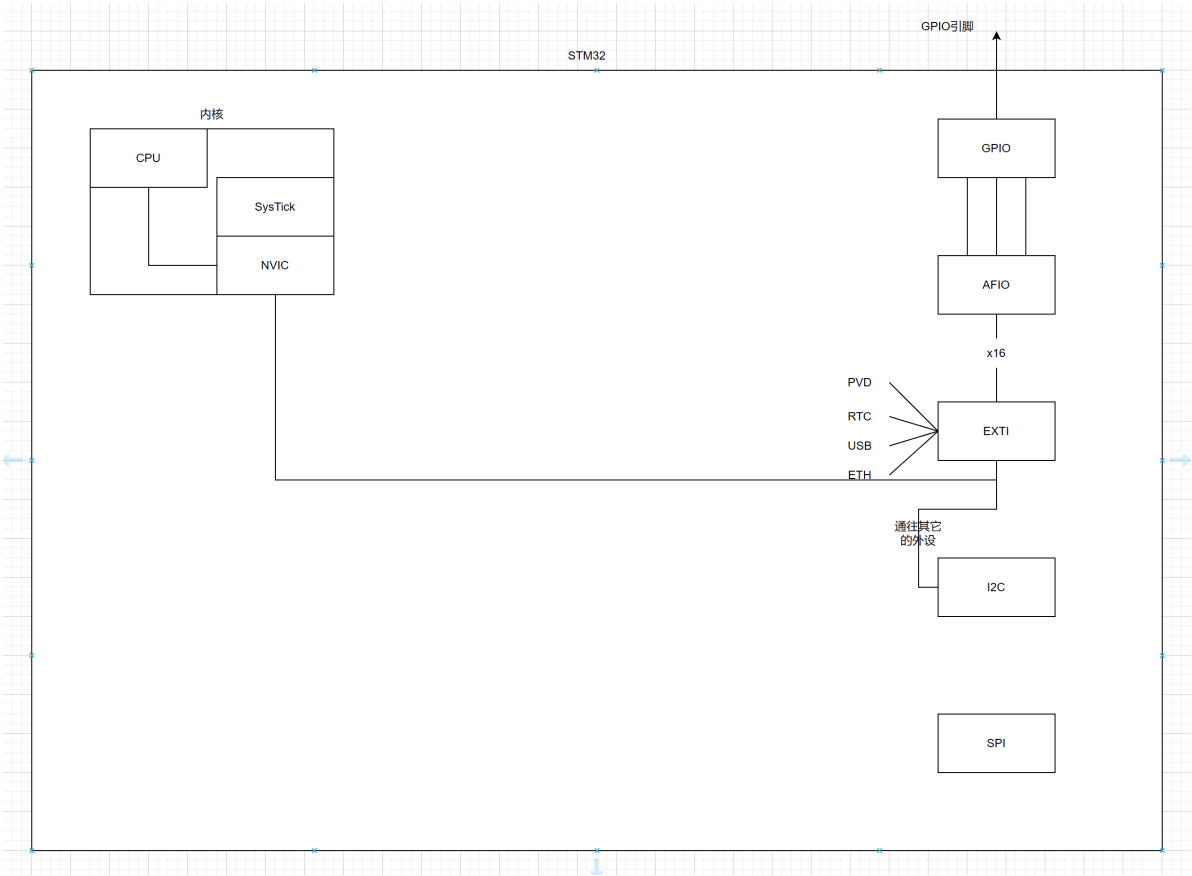
EXTI通道数： `16` 个GPIO_Pin，外加 `4` 个【PVD(电源电压检测)输出、RTC(实时时钟)闹钟、USB唤醒、以太网唤醒】

外部中断的触发响应方式包括中断响应和事件响应。中断响应是正常的流程，引脚电平触发中断；事件响应不会触发中断，而是触发别的外设操作，属于外设之间的联合工作。

2、基本结构

外部中断的整体结构图





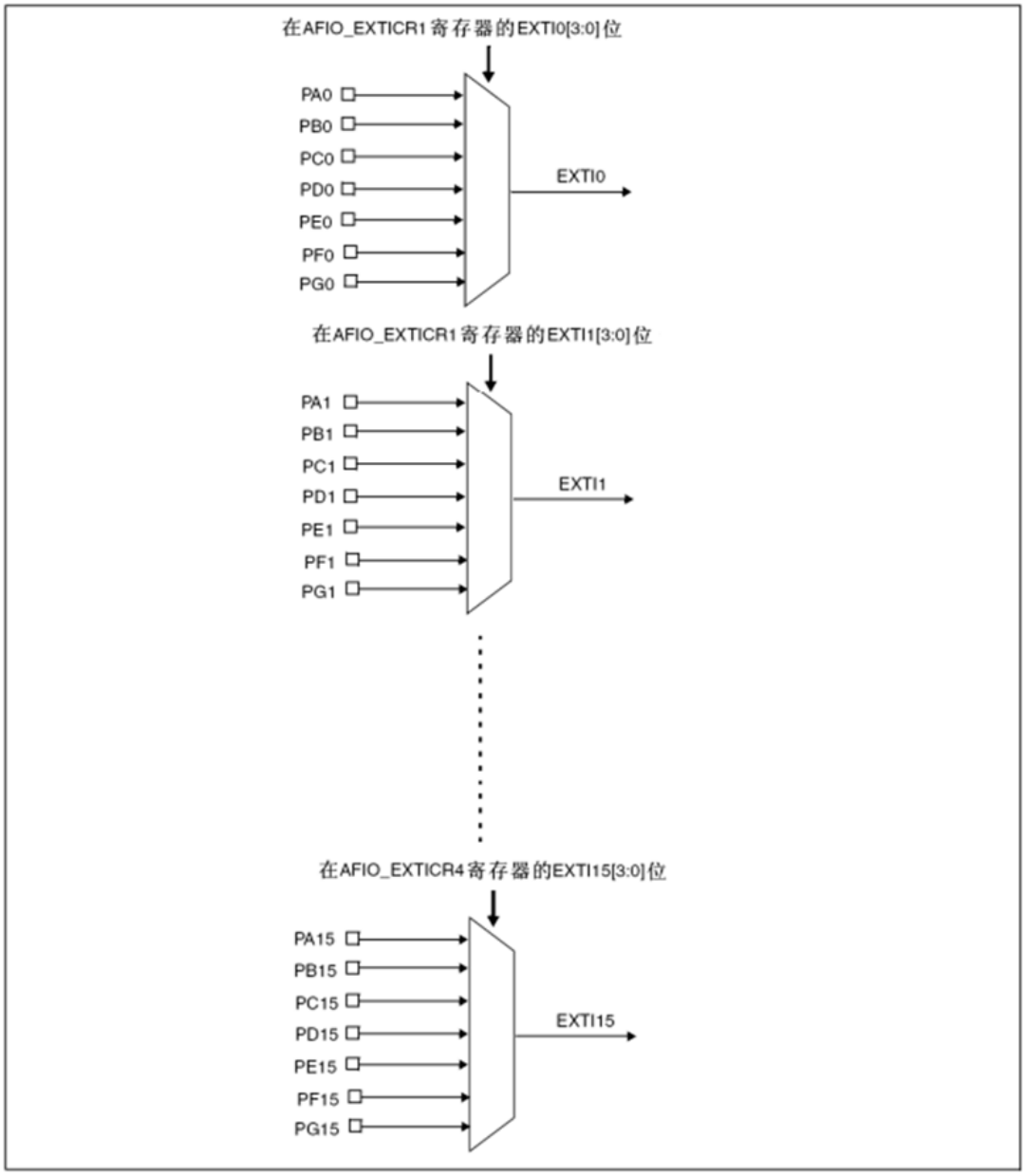
图中最左边是GPIO口的外设，比如GPIOA、GPIOB、GPIOC等等。每个GPIO外设有16个引脚。前面讲过EXTI模块只有16个GPIO的通道，但这里每个GPIO外设都有16个引脚，如果每个引脚占用一个通道，那EXTI的16个通道显然就不够用了，所以在这里会有一个AFIO中断引角选择的电路模块。这个AFIO就是一个数据选择器，它可以在前面三个GPIO外设的16个引脚里选择其中一个，连接到后面的EXTI的通道里。因此前面强调相同的Pin不能同时触发中断。对于PA0、PB0、PC0这些，通过AFIO选择之后只有其中一个能接到EXTI的通道0上，同理PA1、PB1、PC1这些也只能有一个接到通道1上，这就是所有GPIO口都能触发中断，但相同的Pin不能同时触发中断的原因。然后通过AFIO选择之后的16个通道就接到了EXTI边缘检测及控制电路上。同时下面这四个属于蹭网性质的外设也会并列进来，这些加起来
就组成了EXTI的20个输入信号。然后经过EXTI电路之后，分为了两种输出，其中上面接到了NVIC用来触发中断，下面还有20条输出线路接到了其他外设，这就是用来触发其他外设操作的，也就是前面讲的事件响应。
大家注意一下，上面接入NVIC的本来应该是20路中断的输出，但是可能ST公司觉得这20个输出太多了，比较占用NVIC的通道资源，所以把其中外部中断的9-5和15-10给分别分配到了一个通道里。也就是说，外部中断的9-5会触发同一个中断，15-10也会触发同一个中断。因此在后面编程的时候，需要在这两个中断函数里再根据标志位来区分到底是哪个中断进来的。

3、AFIO复用IO口

AFIO主要用于引脚复用功能的选择和重定义。

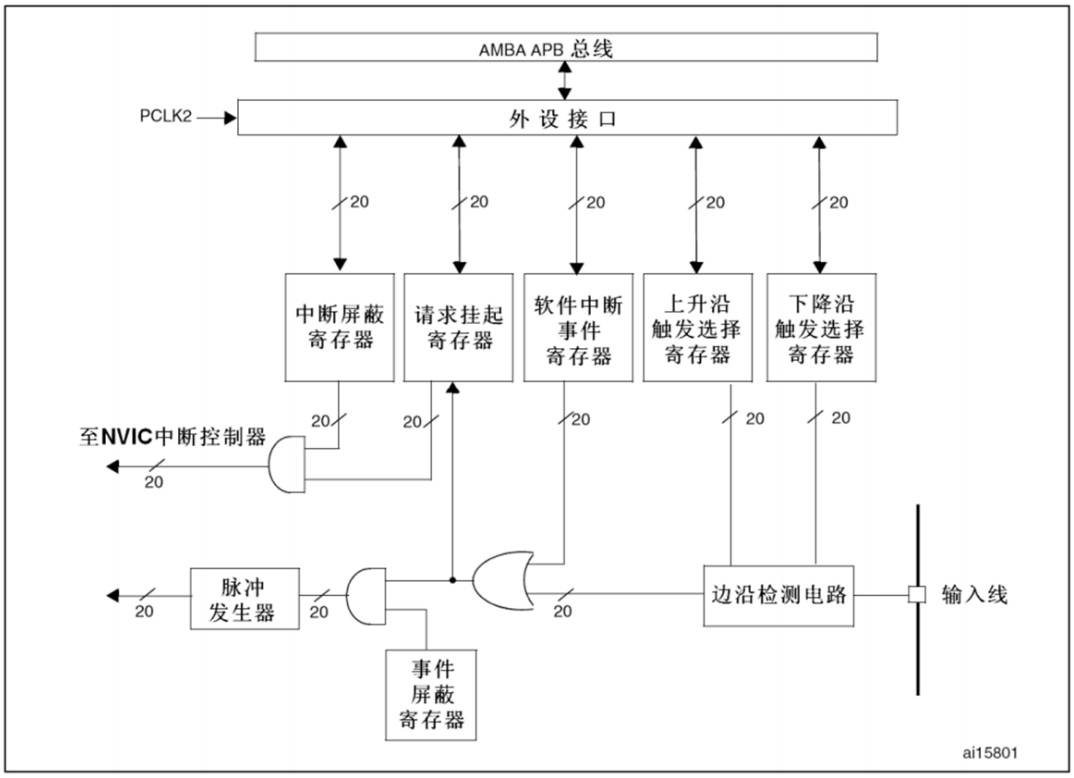
在STM32中，AFIO主要完成两个任务：**复用功能引脚重映射**、**中断引脚选择**。

图20　外部中断通用I/O映像



4、EXTI框图

图19　外部中断/事件控制器框图



三、硬件介绍

1、槽型光耦器件

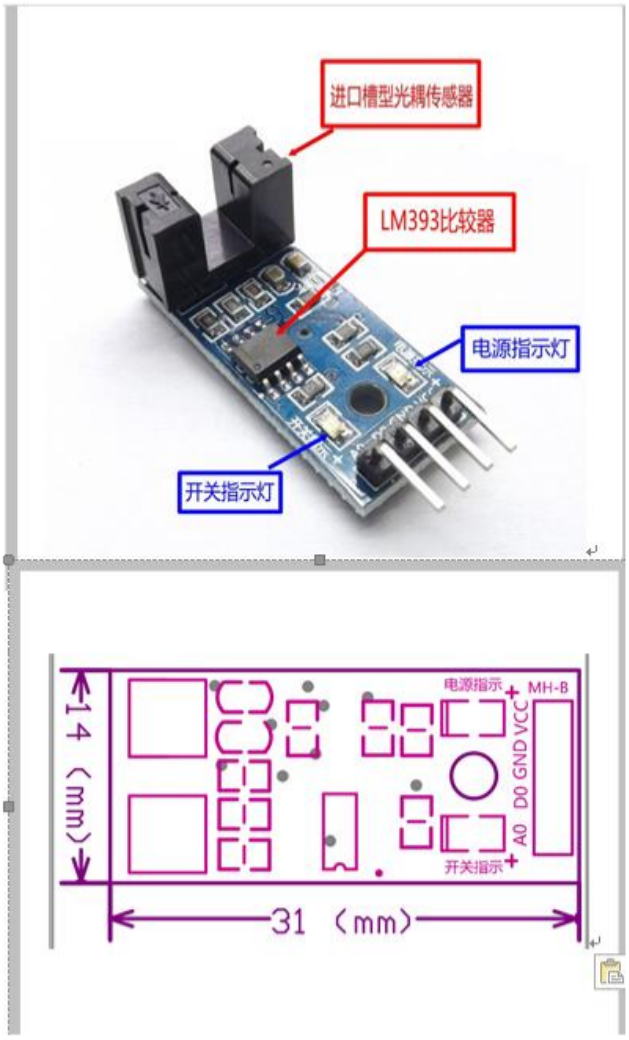
产品特性：

使用ITR9606高灵敏度槽型光耦器件，它由一个红外发光二极管和一个NPN光电三极管组成，槽宽度为5mm。传感器特设M3固定安装孔，调节方向与固定方便易用，使用宽电压LM393比较器，信号干净，波形好，驱动能力强，超过15mA。广泛用于电机转速检测，脉冲计数,位置限位等。

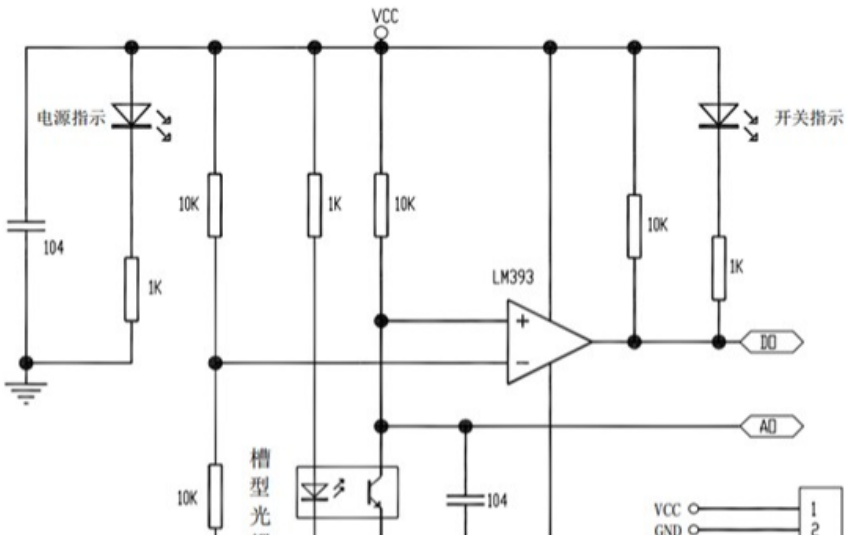
产品参数：

工作电压 3.3V-5V

输出形式 ：数字开关量DO输出（0 和 1），模拟输出AO引脚无效



使用说明：





槽宽：5mm

接好VCC和GND，模块电源指示灯会亮，模块槽中无遮挡时，接收管导通，模块DO输出低电平，开关指示灯亮；遮挡时，DO输出高电平，开关指示灯灭。模块DO可与继电器相连，组成限位开关等功能，也可以与有源蜂鸣器模块相连，组成报警器。DO输出接口可以与单片机IO口直接相连，一般接外部中断，检测传感器是否有遮档，如用电机码盘则可检测电机的转速。

程序设计：

测速模块的 OUT 口链接 MCU 的外部中断口，每当有红外射线导通就是一个外部缓冲。

逻辑设计：

测距离：

测速传感器输出为脉冲信号，一个脉冲中断一次；红外射线导通的时候是低电平，所以我们设置中断为低电平触法模式。一般码盘上有整数格子，无论是多少格其实原理一样，例如 10 格码盘，也就是有10个空格子，电机转一圈后便是射线导通10次，外部低电平触法10次；安装上面的思路，我们的测速传感器就可以发挥出效果了，我们知道一圈就有10个中断，于是我们计算中断次数，得到的总次数除于10也就是电机转动次数了，然后按照轮子的周长，计算轮子一圈是多长，就可以推算出小车已经跑多远了。
测速度：

按照测距离的思路，我们用一个 MCU 定时器计算，1秒内接收多少个外部中断，例如一秒内接收了20个外部中断，我们就可以判断小车速度为1秒小车轮子转两圈，然后再计算出小车轮子的周长，就可得知小车1秒行驶的速度。

注意事项：

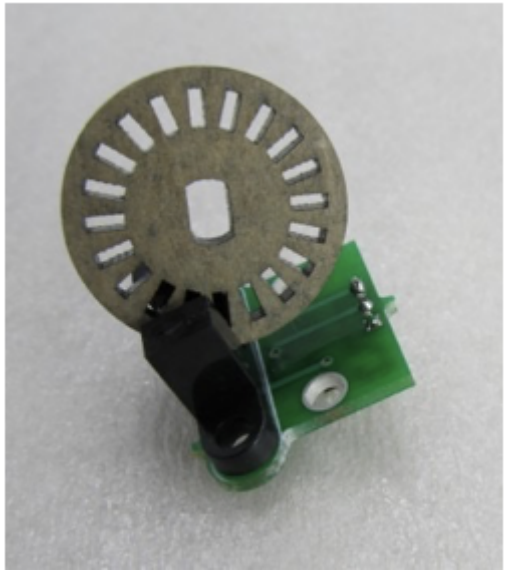
正确接线！切勿将正负接反，使板子电子器件烧毁。Raspberry Pi
玩家应该设置MCU 的I/O口为输入模式/接收模式，否则无法使用。其他 MCU，或者更为高级的控制板如 ARM 这些，若需设置I/O口为输入输出模式，都必须设置为输入模式/接收模式，否则无法使用。51 系列单片机可直接使用，无需设置输入输出模式。

2、正交编码器

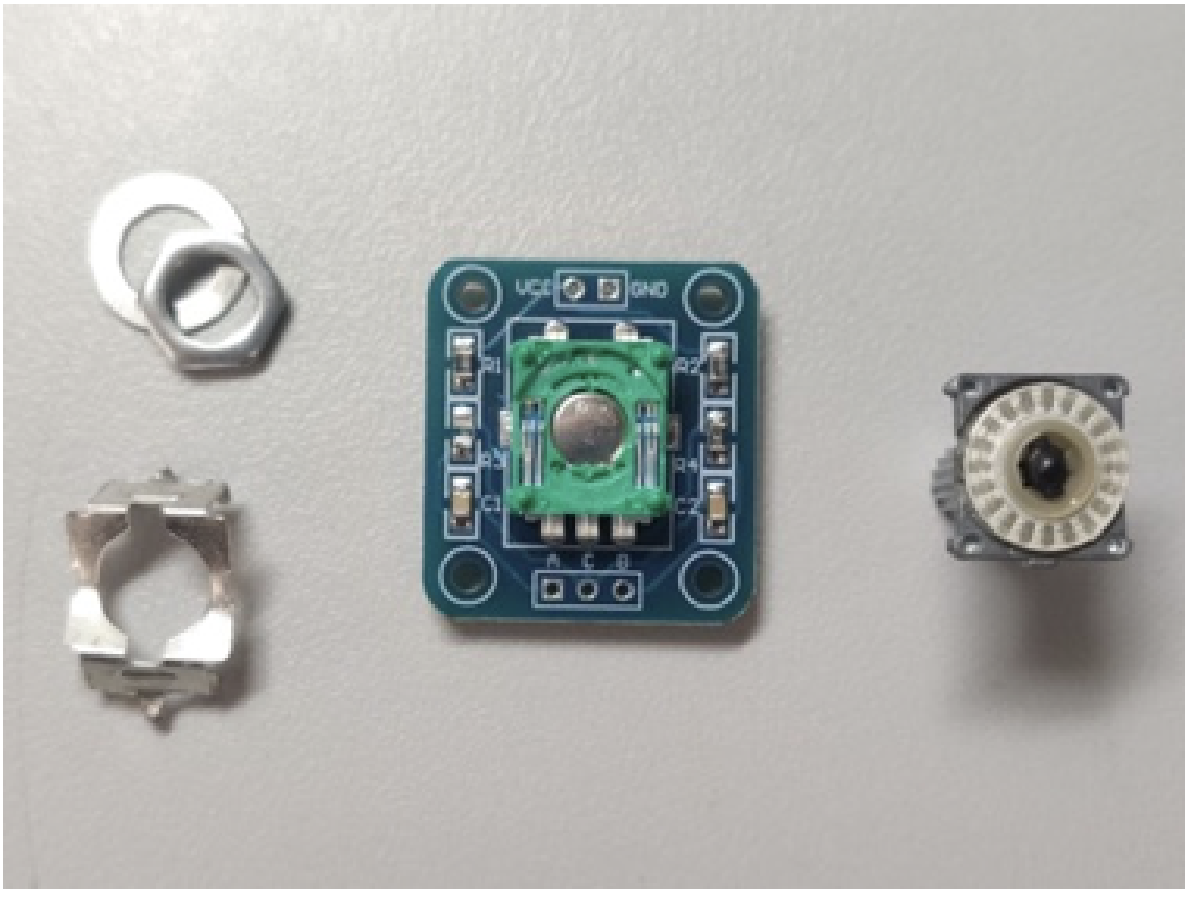
•旋转编码器：用来测量位置、速度或旋转方向的装置，当其旋转轴旋转时，其输出端可以输出与旋转速度和方向对应的方波信号，读取方波信号的频率和相位信息即可得知旋转轴的速度和方向

•类型：机械触点式/霍尔传感器式/光栅式

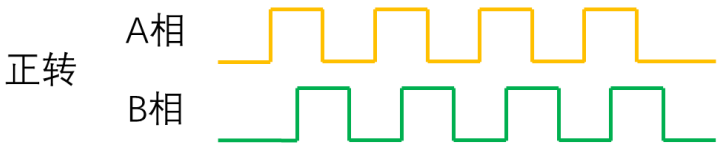
> 单向波形不能区别方向，只能识别速度
>
> 正交波形可以区别方向和速度



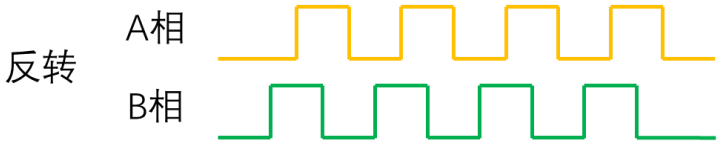

VCC
GND


VCC
GND

下面是正交波形图



边沿	另一相状态
A相↑	B相低电平
A相↓	B相高电平
B相↑	A相高电平
B相↓	A相低电平



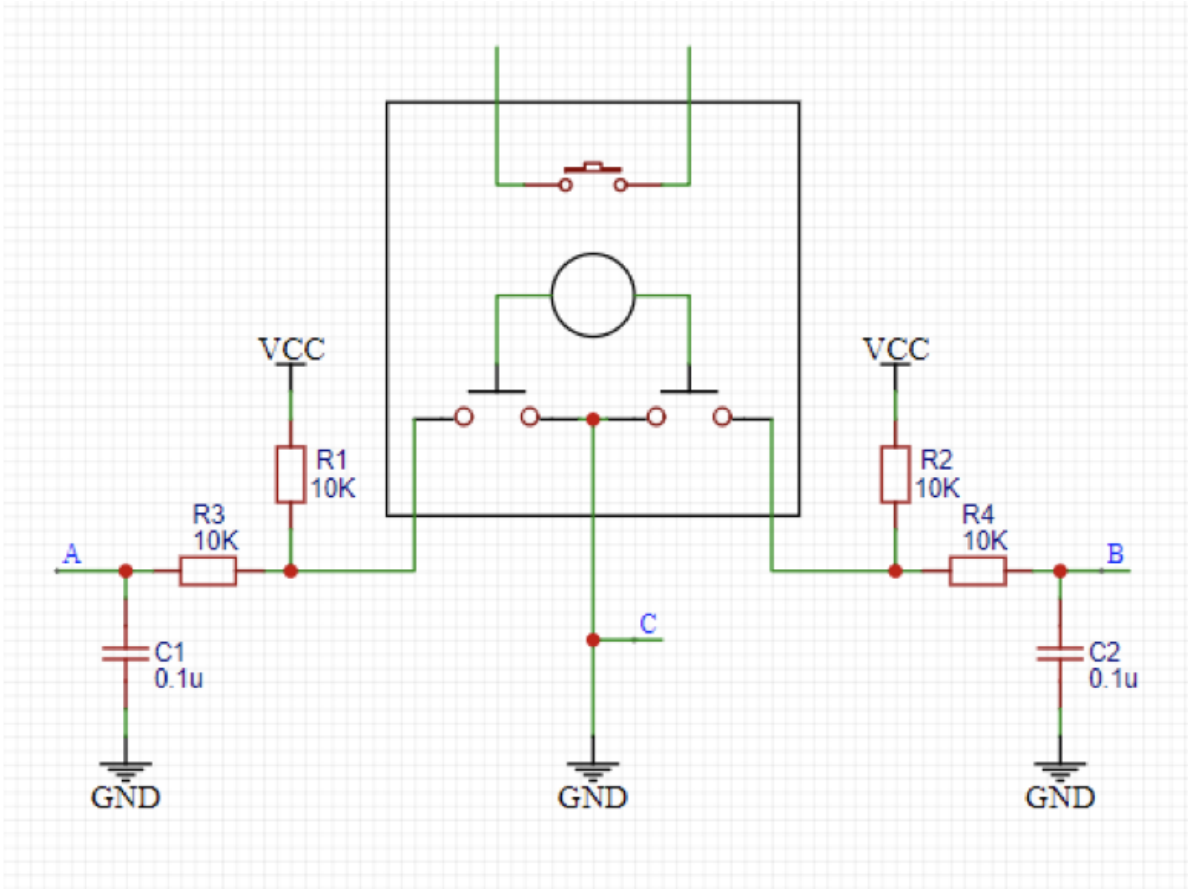
边沿	另一相状态
A相↑	B相高电平
A相↓	B相低电平
B相↑	A相低电平
B相↓	A相高电平

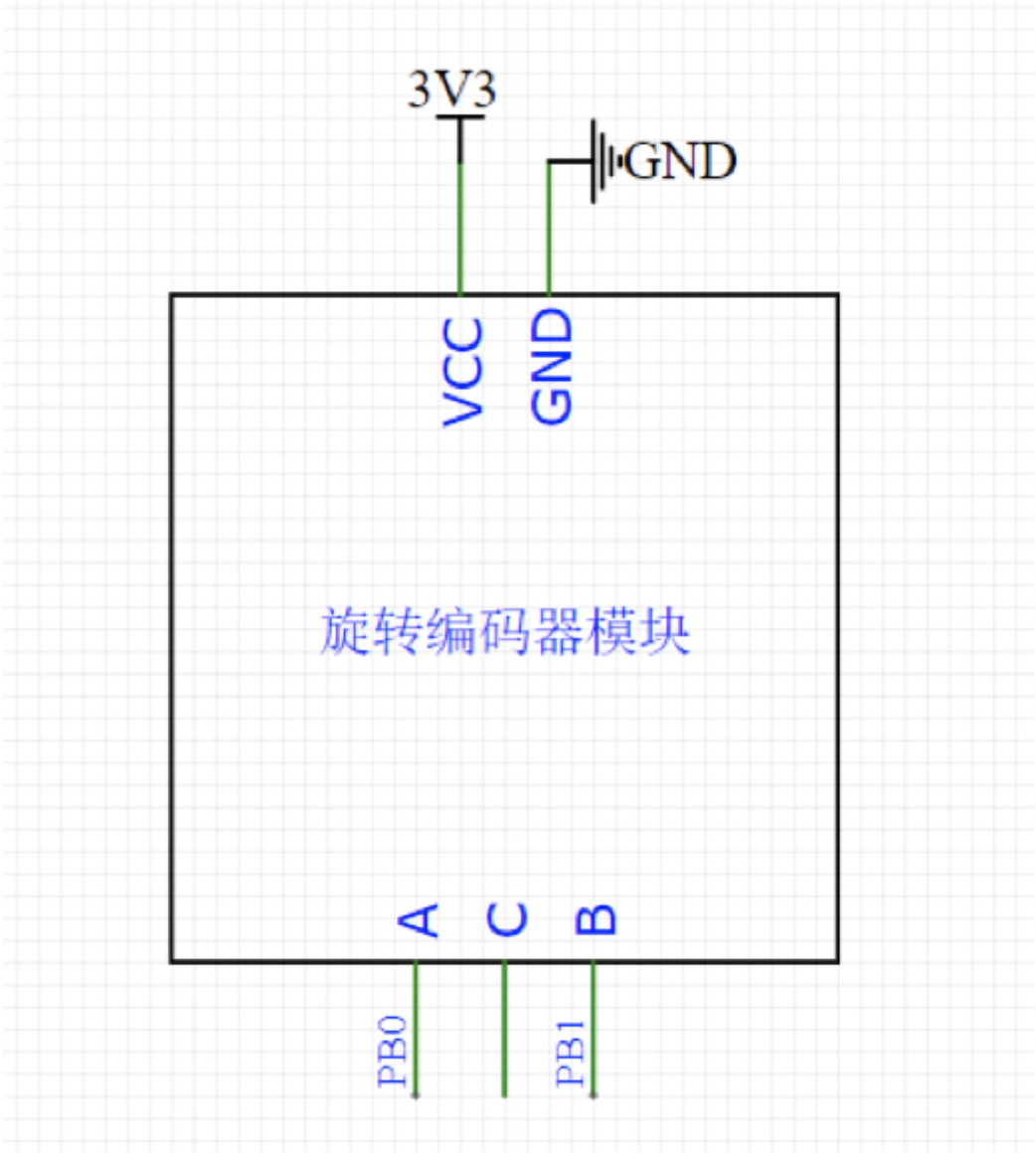
正交编码器一般可以测量位置或者带有方向的速度值，它一般有两个信号输出引脚，一个是A相，一个是B相。

正交信号是如何计次和区分旋转方向呢？首先我们观察一下波形的特点，在正转的时候，第一个时刻A相上升沿，对应B相此时是低电平；继续第2个时刻B相上升沿，对应A相高电平；继续第3个时刻A相下降沿，对应B相高电平；最后是B相下降沿，对应A相低电平；再然后就是A相上升沿，B相低电平了，此时和第一个状态重复了。所以在正转的时候，总结了右边这个表。当出现这些边沿时，对应另一相的状态是表中的四种状态。那么反转的情况呢？第一个时刻B相上升沿，对应A相低电平；第二个时刻A相上升沿，对应B相高电平；第三个时刻B相下降沿，对应A相高电平；第四个时刻A相下降沿，对应B相低电平，这四种状态可以列出下面反转的表。
可以发现，当A、B相出现这些边沿时，对应逆向的状态，正转和反转正好是相反的。比如A相上升沿时，正转B相就是低电平，反转B相就是高电平。其它的也类似，大家可以下来自己再总结。
由于上面的原因，编码器接口的设计逻辑就是首先把A相和B相的所有边沿作为计数器的计数时钟，出现边沿信号时，就计数自增或自减。是增还是减，这个计数的方向由另一相的状态来确定。当出现某个边沿时，判断另一相的高低电平，如果对应另一相的状态出现在上面正转的表里，那就是正转计时自增；反之，另一相的状态出现在下面反转的表里，那就是反转计时自减。

4、平衡小车电机



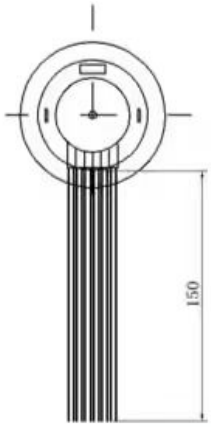

VCC
R1
10K
R3
10K
A
C1
0.1u
GND
C
GND
VCC
R2
10K
R4
10K
B
C2
0.1u
GND


3V3
GND
VCC
GND
旋转编码器模块
A
C
B
PB0
PB1



编码器的连接:

红：电机电源+（和白色对调可以正反转）
黑：编码器电源-
黄：信号线 电机一圈11个脉冲
绿：信号线 分辨率11*减速比=分辨率
蓝：编码器电源+
白：电机电源-（和红色对调可以正反转）



输出信号： 方波（默认）/
正弦波（定制） AB相
电源电压范围：3.3V-5V
工作温度范围：-40℃~120℃

电机相关参数名称含义说明：

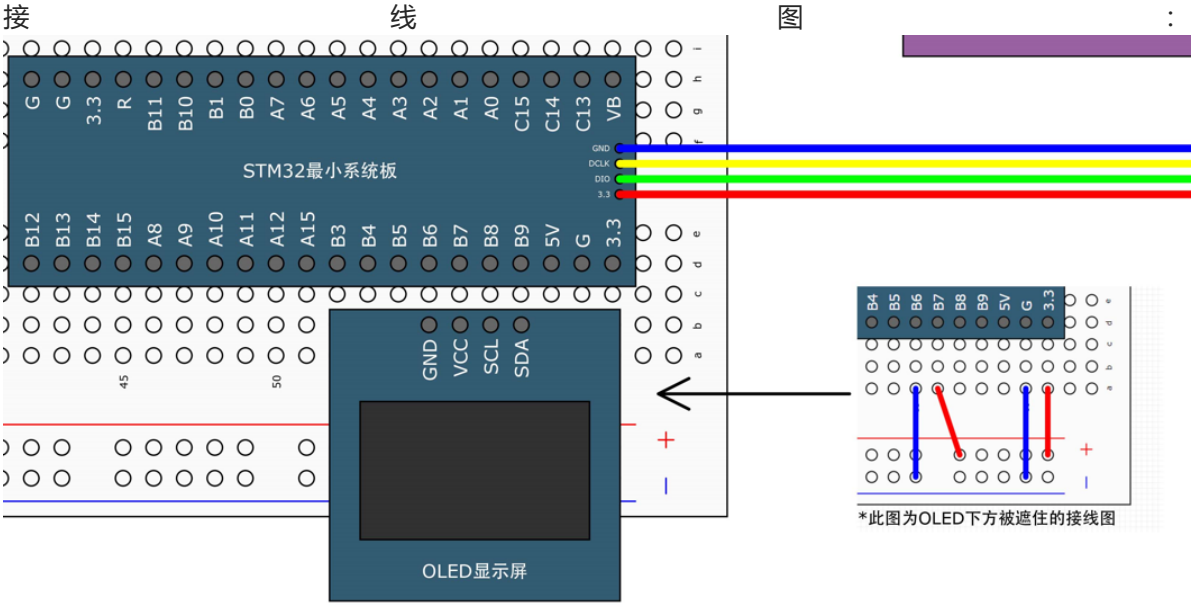
1.空载转速：电机正常通电无负载状态的转速。(单位rpm或 转/分钟或r/min)
2.空载电流：电机正常通电无负载状态的电流。(单位：MA毫安）
3.负载力矩：电机负载测试时候的额定扭矩，仅用于测试参考；（单位： gcm 克每厘米或kgcm 公斤每厘米）
4.负载转速：电机在负载力矩下的转速。（单位：rpm或 转/分钟或r/min)
5.负载电流：电机在负载力矩下的电流。（单位：mA毫安或A安）
6.堵转力矩：又叫启动扭力，为电机所能承受的最大扭力标准，超过该扭力，电机将停转或堵转。（单位：gcm 克 明每厘米或kgcm公斤每厘米）
7.堵转电流：也叫启动电流，为电机遭到堵转停止时候的最大电流；（单位：mA海毫安或A安）

四、OLED

OLED（Organic Light Emitting Diode）：有机发光二极管

我们可以用显示屏来显示我们的计数值变量，这样方便我们观察程序运行状态和调试程序。

接 线 图 ：



使用写好的库函数：

函数	作用
OLED_Init();	初始化
OLED_Clear();	清屏
OLED_ShowChar(1, 1, 'A');	显示一个字符
OLED_ShowString(1, 3, "HelloWorld!");	显示字符串
OLED_ShowNum(2, 1, 12345, 5);	显示十进制数字
OLED_ShowSignedNum(2, 7, -66, 2);	显示有符号十进制数字
OLED_ShowHexNum(3, 1, 0xAA55, 4);	显示十六进制数字
OLED_ShowBinNum(4, 1, 0xAA55, 16);	显示二进制数字

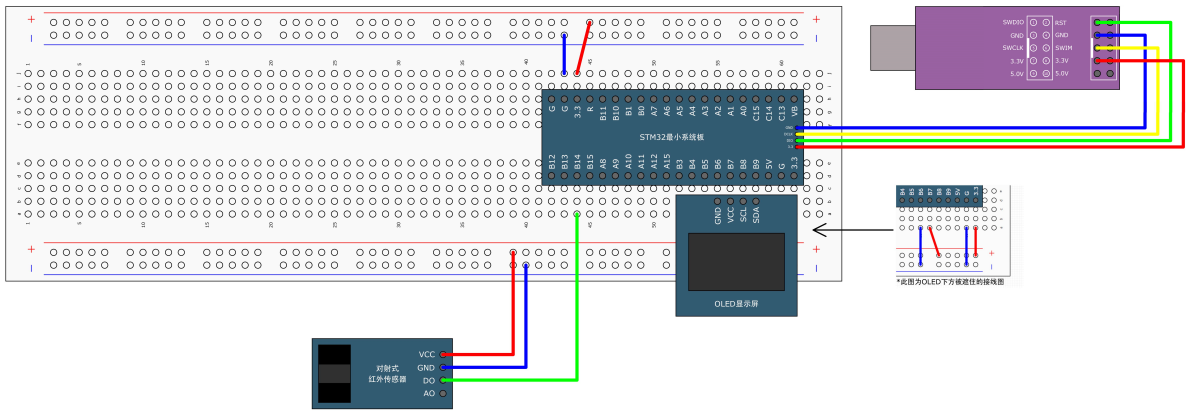
五、EXTI外部中断实验

1、EXTI外部中断计数

（1）实验目的

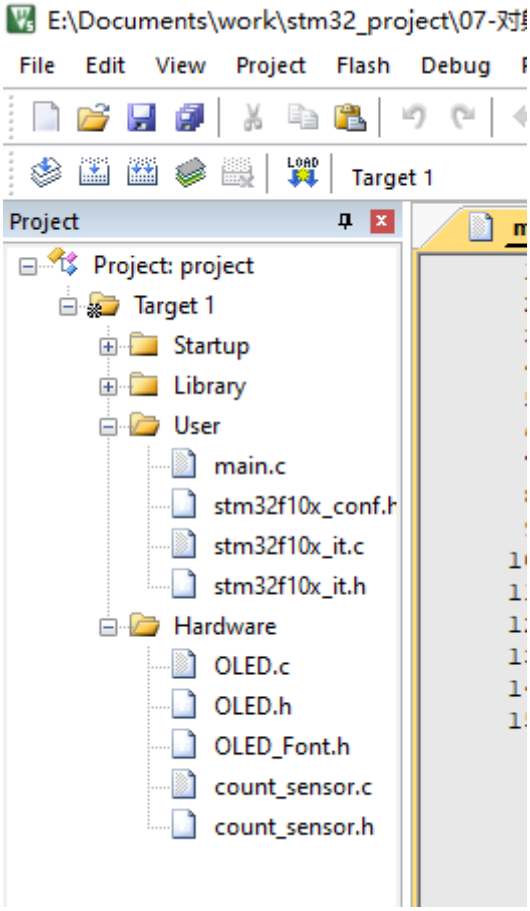
通过对STM32的EXTI外部中断的检测实现使用挡光片多次遮挡对射式红外传感器的信号从而让OLED上显示出遮挡次数信息。

（2）硬件电路接线图



（3）程序设计

工程结构如下图：



其中Hardware组与之前工程不一样，这个组主要提供了OLED和红外传感器驱动方法，其中OLED的驱动本次课程仅做使用，暂时不展开讲解。具体代码展示如下：

count_sensor.c

```c
#include "stm32f10x.h"                  // Device header

uint16_t CountSensor_Count;

void CountSensor_Init(void) {
    RCC_APB2PeriphClockCmd(RCC_APB2Periph_GPIOB, ENABLE);    // 初始化GPIOB的时钟
    RCC_APB2PeriphClockCmd(RCC_APB2Periph_AFIO, ENABLE);     // 初始化AFIO的时钟

    GPIO_InitTypeDef GPIO_InitStructure;
    GPIO_InitStructure.GPIO_Mode = GPIO_Mode_IPU;    // 配置上拉输入
    GPIO_InitStructure.GPIO_Pin = GPIO_Pin_14;  // 配置传感器信号检测引脚
    GPIO_InitStructure.GPIO_Speed = GPIO_Speed_50MHz;
    GPIO_Init(GPIOB, &GPIO_InitStructure);  // 初始化GPIOB

    // EFIO和NVIC时钟默认开启，不需要初始化
    // 设置AFIO选择器选通
    GPIO_EXTILineConfig(GPIO_PortSourceGPIOB, GPIO_PinSource14);

    EXTI_InitTypeDef EXTI_InitStructure;
    EXTI_InitStructure.EXTI_Line = EXTI_Line14;
    EXTI_InitStructure.EXTI_LineCmd = ENABLE;
    EXTI_InitStructure.EXTI_Mode = EXTI_Mode_Interrupt; // 配置EXTI中断响应触发模式
    EXTI_InitStructure.EXTI_Trigger = EXTI_Trigger_Falling; // 配置EXTI下降沿触发
    EXTI_Init(&EXTI_InitStructure);      // 初始化EXTI
```

```
    // NVIC优先级分组整个芯片只需要设置一次，不能重复设置。这里更好的可以放到主函数main.c中。
    NVIC_PriorityGroupConfig(NVIC_PriorityGroup_2);

    NVIC_InitTypeDef NVIC_InitStructure;
    NVIC_InitStructure.NVIC_IRQChannel = EXTI15_10_IRQn;    // 配置NVIC的EXTI15_10通道
    NVIC_InitStructure.NVIC_IRQChannelCmd = ENABLE;
    NVIC_InitStructure.NVIC_IRQChannelPreemptionPriority = 1;   // 配置抢占优先级
    NVIC_InitStructure.NVIC_IRQChannelSubPriority = 1;  // 配置响应优先级
    NVIC_Init(&NVIC_InitStructure);     // 初始化NVIC
}

uint16_t CountSensor_Get(void) {
    return CountSensor_Count; // 获取计次数值
}

// 这个方法CPU会收到中断后自动调用
void EXTI15_10_IRQHandler(void) {
    // 检查中断位是哪一位的中断信号
    if (EXTI_GetITStatus(EXTI_Line14) == SET) {
        CountSensor_Count++;
        // 注意中断处理结束一定要清除中断标记，否则就会不停地进入中断处理程序
        EXTI_ClearITPendingBit(EXTI_Line14);
    }
}
```

count_sensor.h

```
# ifndef __COUNT_SENSOR_H
# define __COUNT_SENSOR_H

void CountSensor_Init(void);
uint16_t CountSensor_Get(void);

# endif
```

main.c

```
#include "stm32f10x.h"                  // Device header
#include "OLED.h"
#include "count_sensor.h"

int main(void){

    OLED_Init();
    CountSensor_Init();
    OLED_ShowString(1, 1, "Count:");    // 设置OLED第一行第一列显示检测到的计次变量标题

    while(1){
```

```
        OLED_ShowNum(1, 7, CountSensor_Get(), 5);    // 在OLED从第一行第七列开始显示计次数
据，数值宽度5个字符
    }
}
```

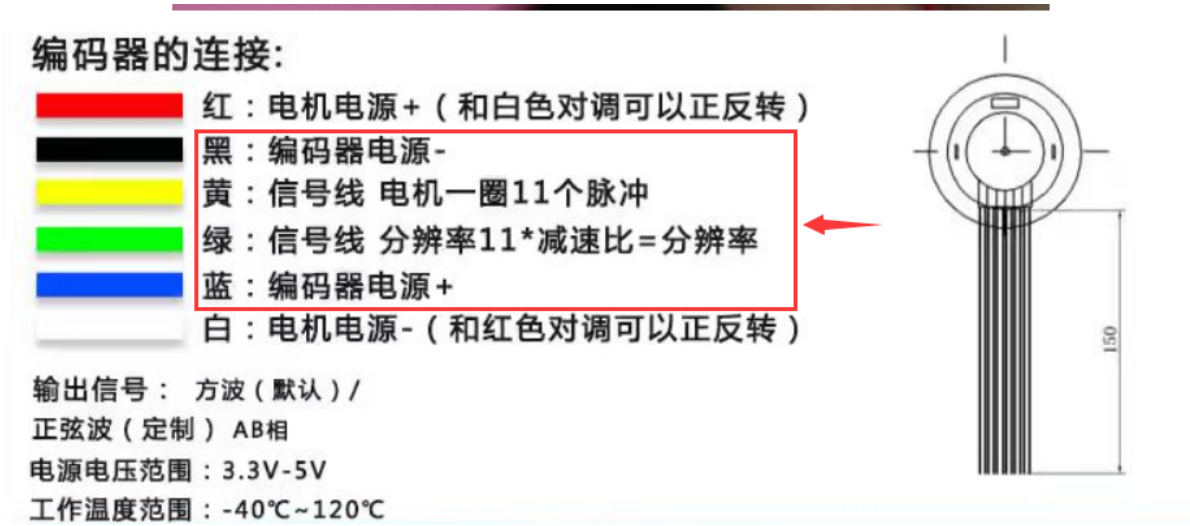
（4）代码完成后，编译并下载到硬件中执行，会发现实验结果是用挡光片来回遮挡传感器信号，OLED计次正确显示，符合预期设计。

2、EXTI外部中断计数电机正反转数

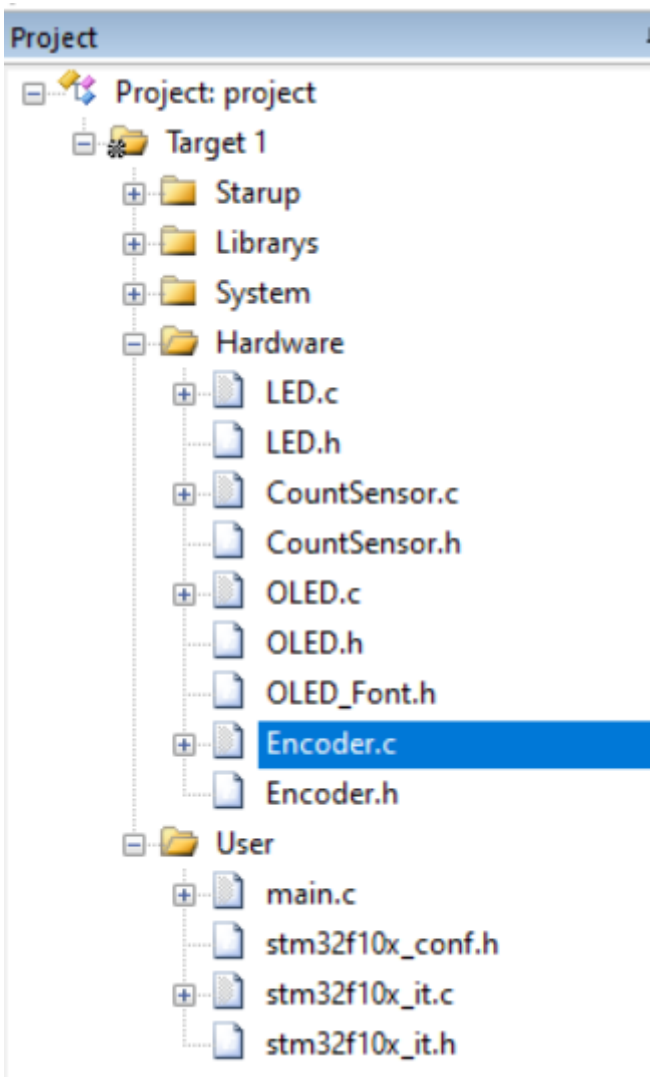
（1）实验目的

通过对STM32的EXTI外部中断检测正反转及转数

（2）硬件电路接线图



（3）程序设计



Encoder.c

```
#include "stm32f10x.h"                  // Device header

int16_t Encoder_Count;

void Encoder_Init(void)
{
    //1.开启GPIO输入

    RCC_APB2PeriphClockCmd(RCC_APB2Periph_GPIOA,ENABLE);

    GPIO_InitTypeDef GPIO_InitTypeStructure;
    GPIO_InitTypeStructure.GPIO_Mode = GPIO_Mode_IPD; //下拉输入,和设备默认一致。
    GPIO_InitTypeStructure.GPIO_Pin = GPIO_Pin_5 | GPIO_Pin_6; //引脚选择
    GPIO_InitTypeStructure.GPIO_Speed = GPIO_Speed_50MHz;
    GPIO_Init(GPIOA,&GPIO_InitTypeStructure);

    //2.AFIO选择
    //开启AFIO时钟
    RCC_APB2PeriphClockCmd(RCC_APB2Periph_AFIO,ENABLE);
    //线路选择
    GPIO_EXTILineConfig(GPIO_PortSourceGPIOA,GPIO_PinSource5);
```

```
    //3.EXTI边缘检测及控制
    EXTI_InitTypeDef EXTI_InitTypeStructure;
    EXTI_InitTypeStructure.EXTI_Line = EXTI_Line5 | EXTI_Line6; // 线路5 和 6
    EXTI_InitTypeStructure.EXTI_LineCmd = ENABLE; //使能
    EXTI_InitTypeStructure.EXTI_Mode = EXTI_Mode_Interrupt; //中断，不是事件
    EXTI_InitTypeStructure.EXTI_Trigger = EXTI_Trigger_Rising;//上升沿

    EXTI_Init(&EXTI_InitTypeStructure);


    //4.NVIC
    NVIC_PriorityGroupConfig(NVIC_PriorityGroup_2);//分组，全局设置一次即可。

    NVIC_InitTypeDef NVIC_InitTypeSturcture;
    NVIC_InitTypeSturcture.NVIC_IRQChannel = EXTI9_5_IRQn;
    NVIC_InitTypeSturcture.NVIC_IRQChannelCmd = ENABLE;
    NVIC_InitTypeSturcture.NVIC_IRQChannelPreemptionPriority = 2; // 抢占优先级
    NVIC_InitTypeSturcture.NVIC_IRQChannelSubPriority =2; //响应优先级
    NVIC_Init(&NVIC_InitTypeSturcture);
}



void EXTI9_5_IRQHandler(void)
{

    if(EXTI_GetITStatus(EXTI_Line5)==SET)
    {
        if(GPIO_ReadInputDataBit(GPIOA,GPIO_Pin_6)==0) //判断PA6的电平
        {
            Encoder_Count++;    //正转
        }
        //清除中断标识
        EXTI_ClearITPendingBit(EXTI_Line5);
    }


    if(EXTI_GetITStatus(EXTI_Line6)==SET)
    {
        if(GPIO_ReadInputDataBit(GPIOA,GPIO_Pin_5)==0) //判断PA5的电平
        {
            Encoder_Count--;    //反转
        }
        //清除中断标识
        EXTI_ClearITPendingBit(EXTI_Line6);
    }



}
```

```
int16_t Encoder_CountGet(void)
{
    return Encoder_Count;
}
```

Encoder.h

```
#ifndef __ENCODER_H
#define __ENCODER_H
int16_t Encoder_CountGet(void);
void Encoder_Init(void);

#endif
```

main.c

```
#include "stm32f10x.h"                  // Device header
#include "LED.h"
#include "OLED.h"
#include "Encoder.h"

uint16_t Count;

int main(void)
{
    LED_Init();
    OLED_Init();
    Encoder_Init();

    while(1)
    {
        OLED_ShowSignedNum(1,1,Encoder_CountGet(),5);
    }
}
```

课程小结

1. STM32的中断包含EXTI、TIM等多种，它们统一由NVIC管理，有了NVIC就可以更好的节约CPU的资源。
2. EXTI主要自动检测GPIO端口上的电压变化，一旦发生变化立刻会申请中断，通过这一特性可以对IO输入信号进行快速检测并作出处理。

随堂作业

1. 请说说EXTI是怎么回事?
2. 根据编码器，旋转编码器，顺转（大于0）越多流水灯越快，反转（减小值）则越慢；反转（小于0）反向亮，值越小流水灯越快。